2026학년도 졸업프로젝트

최 종 보 고 서

스케줄링 알고리즘을 활용한

다중 사용자 LLM API 요청 관리 시스템

학 과	홍익대학교 컴퓨터공학과
학 번	C235180
성 명	서민지
지도교수	이장호 교수님

2026년 5월

홍익대학교 컴퓨터공학과

1. 서론

1.1 연구 배경

ChatGPT, Claude, Gemini 등 대규모 언어 모델(Large Language Model, LLM) 서비스는 주로 API(Application Programming Interface) 형태로 제공된다. 개별 사용자가 직접 가입해 쓰기도 하지만, 기업이나 연구실에서는 조직 계정 하나를 여러 구성원이 공유하거나 자체 LLM 서버를 운영하는 사례가 늘고 있다.

다중 사용자 환경에서는 요청을 어떻게 관리할지가 문제이다. OpenAI는 조직(Organization) 단위로 분당 요청 수·토큰 수 한도를 두고 [1], Anthropic은 사용 등급별 호출 한도를 적용한다 [2]. 한도는 조직 단위로 묶여 있으므로, 구성원끼리 그것을 어떻게 나누어 쓸지는 운영자가 따로 정해야 한다. 자체 LLM 서버도 동시 처리 수가 제한되어 같은 문제가 생긴다. 별도 관리 기능이 없으면 도착 순서대로 처리하거나 호출 횟수만 제한하는 수준에 머무른다. 그러면 긴급한 요청이 뒤로 밀리고, 사용자별 차등 배분도 어려워진다.

1.2 연구 목적

운영체제 수업에서 배운 프로세스 스케줄링 알고리즘을 LLM API 요청 관리에 적용하고, 알고리즘별 성능과 공정성을 비교하는 것이 본 프로젝트의 목적이다. 구체적으로 다음 세 가지 연구 질문에 답한다.

1. **RQ1 (스케줄링 적용)**: FCFS, Priority Scheduling, MLFQ, WFQ를 LLM API 환경에 맞게 구현할 수 있는가?
2. **RQ2 (성능 비교)**: 평균 대기시간·처리량 등에서 알고리즘 간 효율성 차이가 있는가?
3. **RQ3 (공정성 분석)**: Jain's Fairness Index(JFI, 자원 배분의 균등성을 0~1 사이 점수로 나타내는 지표)로 알고리즘별 공정성은 어떻게 나타나는가?

1.3 연구 범위

- **대상 알고리즘**: FCFS, Priority Scheduling, MLFQ, WFQ 네 가지 (OS·네트워크 분야 대표 방식).
- **실험 방식**: 시간 기반 시뮬레이션(알고리즘 특성 비교)과 Ollama를 활용한 실서버 실험 두 가지.
- **지표**: 성능에는 평균 대기시간과 처리량(req/s, 초당 처리한 요청 수)을, 공정성에는 JFI를 사용한다.
- **Rate Limiter**: 요청 진입 한도를 관리하는 전처리 장치로, 스케줄링 비교 대상이 아닌 별도 단계로 구현하였다.

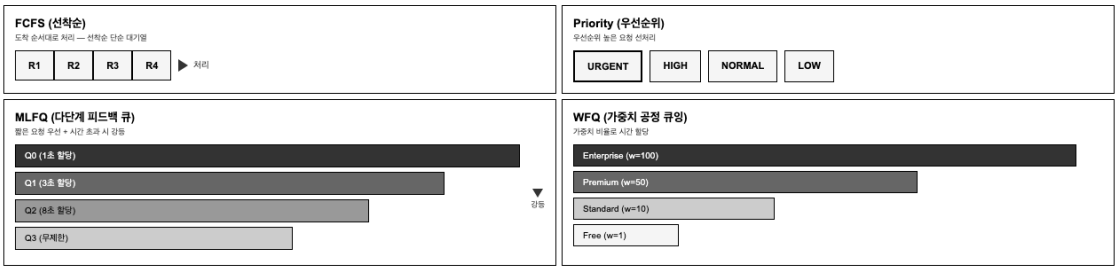
2. 관련 연구

이 장에서 다루는 내용은 세 가지이다. 본 프로젝트에서 사용한 스케줄링 알고리즘의 이론, 기존 LLM 서빙 도구의 동작 방식, 그리고 자원 배분의 공정성을 측정하는 방법이다.

2.1 스케줄링 알고리즘

프로세스 스케줄링은 운영체제에서 CPU 자원을 여러 프로세스에 나누어 주는 방법이다 [3]. 본 프로젝트에서 사용한 네 가지 알고리즘의 핵심 특성은 그림 1과 같다.

그림 1. 스케줄링 알고리즘 개념 비교



[그림 1] 스케줄링 알고리즘 개념 비교

FCFS(First-Come, First-Served)는 도착 순서대로 처리하는 가장 단순한 스케줄링 방식이며, 운영체제의 일괄 처리 시대부터 사용된 고전 기법이다. 새 요청이 도착하면 대기열의 맨 뒤에 추가되고, 처리는 항상 맨 앞 요청부터 시작된다. 한 요청이 처리되는 동안 다른 요청은 끼어들 수 없으며 비선점형으로만 동작한다. 구현이 매우 쉽고 별도 자료구조도 필요 없다. 어느 요청이든 차례가 오기 마련이므로 기아(Starvation, 특정 요청이 영원히 처리되지 못하는 현상)는 일어나지 않는다. 다만 긴 요청이 앞에 있으면 뒤에 있는 짧은 요청들이 모두 지연되는 호위 효과(Convoy Effect)가 생긴다 [3]. 우선순위나 등급은 고려하지 않으므로 긴급 요청을 따로 빠르게 처리하지 못한다. 이 알고리즘은 다른 세 알고리즘과 비교할 때 기준선 역할을 맡는다.

Priority Scheduling은 각 요청에 우선순위를 부여하여 긴급한 요청을 먼저 처리하는 알고리즘이다. 운영체제에서는 시스템 프로세스나 인터럽트 처리에 우선권을 주기 위해 폭넓게 사용된다. 대기열에서 요청을 꺼낼 때 가장 높은 우선순위의 요청을 먼저 선택하며, 같은 우선순위 요청들 사이에서는 도착 순서대로 처리한다. 구현은 우선순위 큐(Priority Queue) 자료구조를 쓰면 단순해진다. 낮은 우선순위 요청은 계속 밀려나 끝내 처리되지 못하는 기아 현상에 빠질 수 있다는 단점이 있다. 이를 막으려고 오래 기다린 요청의 우선순위를 점차 올려주는 에이징(Aging) 기법을 적용하였다 [3]. 단, 긴급도를 사람이 직접 지정해야 하므로 자동 분류는 어렵다. 사용자가 요청마다 긴급도를 지정할 수 있는 LLM API 환경에 잘 맞아 채택하였다.

MLFQ(Multi-Level Feedback Queue)는 여러 단계의 대기열을 두고 작업 특성에 따라 우선순위를 자동으로 조정하는 알고리즘이다. 이 알고리즘의 가장 큰 특징은, 어떤 요청이 짧은지 미리 모르더라도 실행 시간을 관찰해 자동으로 분류한다는 점이다. 새 요청은 최상위 큐에 들어가고, 시간 할당량(Time Quantum, 한 번에 쓸 수 있는 최대 시간) 안에 끝나지 않으면 하위 큐로 내려간다 [3]. 처리할 때는 상위 큐가 비어 있을 때만 하위 큐를 본다. 그래서 짧은 요청은 빠르게 끝나고 긴 요청은 점점 아래로 밀린다. 하위 큐로만 계속 밀리면 기아 상태가 되므로, 일정 주기로 모든 요청을 다시 최상위 큐로 끌어올리는 부스팅(Boosting) 규칙을 두었다. 짧은 요청의 응답 속도가 빠르고 작업 특성을 미리 알 필요가 없어 범용성이 높다는 점이 장점이다. 반면 시간 할당량 단위로 실행을 중단·재개할 수 있어야만 효과가 나타나므로, 추론 중단이 불가능한 LLM 서빙 도구에서는 적용에 제약이 있다. 이론적 효과와 실용적 한계를 모두 살펴보고자 시뮬레이션에서는 선점형·비선점형을 함께 비교하고, 실서버에서는 비선점형만 적용하였다.

WFQ(Weighted Fair Queuing)는 네트워크 라우터에서 한정된 대역폭을 여러 흐름이 공평하게 나누어 쓰도록 만들어진 알고리즘이다 [4]. 본래 목적은 네트워크 트래픽의 QoS(Quality of Service) 보장이며, 인터넷 라우터에 널리 쓰인다. 자원을 완벽히 공평하게 나누는 이상적 모델인 GPS(Generalized Processor Sharing)에 가깝게 동작하도록, 각 요청에 가상 완료 시각(Virtual Finish Time, VFT)을 부여한다. 가상 완료 시각이 가장 작은 요청부터 처리하며, 가중치가 큰 사용자의 VFT는 천천히 증가해 결과적으로 더 자주 처리된다. 가중치 비율에 따라 자원을 비례 배분하므로 사용자별 차등 서비스를 자연스럽게 구현할 수 있다. 처리 중단 없이도 가중치 차등이 작동하므로 비선점형 환경에 적용할 수 있다. 덕분에 Ollama처럼 추론 중단이 불가능한 LLM 서빙 도구에서도 쓸 수 있다. 다만 모든 요청에 대해 VFT를 계산하고 정렬해야 하므로 단순 큐보다 연산 비용이 크다. 구독 등급별 차등 서비스를 구현할 알고리즘으로 사용하였으며, 자세한 VFT 계산은 4.3절에서 다룬다.

2.2 LLM 서빙 시스템

LLM 서빙은 학습된 모델을 실행해 사용자 요청에 응답을 제공하는 것을 뜻한다. LLM은 이전 토큰(Token, LLM이 한 번에 처리·생성하는 텍스트 조각 단위)을 참고해 다음 토큰을 하나씩 생성하며, 매 토큰마다 GPU 자원이 소비된다. GPU 메모리 용량이 동시에 처리할 수 있는 요청 수를 정하므로, 다중 사용자 환경에서는 한정된 GPU 자원을 여러 요청이 경쟁하게 된다. 대표 오픈소스 도구인 vLLM [5]과 Hugging Face TGI(Text Generation Inference) [6]는 여러 요청을 동시 처리하는 연속 배치(Continuous Batching)를 지원해 높은 처리량을 달성한다. 연속 배치는 매 토큰 생성 단계마다 새 요청을 배치에 합류시키는 방식으로, 전체 처리량은 늘리지만 각 요청을 어떤 순서로 끼워 넣을지는 별도 결정이 필요하다. 본 프로젝트 실서버 실험에 사용한 Ollama [10]는 로컬에서 REST API로 LLM을 호출하는 도구로, 한 번에 한 요청씩 순차 처리하고 추론 중단이 불가능하다.

이들 도구는 다중 사용자 환경의 우선순위 관리나 공정성 보장 기능을 따로 두지 않으며, 초점은 전체 처리량을 높이는 데 있다. OpenAI [1]와 Anthropic [2]의 상용 API도 조직 단위의 분당 한도만 제공할 뿐

조직 내부에서 사용자별 차등이나 우선순위 처리는 지원하지 않는다. 결국 "어떤 사용자를 먼저 처리할 것인가"는 서비스 운영자의 몫이며, 본 프로젝트가 이 자리에 스케줄링 이론을 끌어와 적용하였다.

2.3 공정성 측정

자원 배분의 균등성을 측정하는 대표 지표로 Jain's Fairness Index(JFI)가 있다 [7]. JFI는 사용자 수에 무관하게 0~1 사이 값을 제공한다.

$$JFI = (x_1 + x_2 + \dots + x_n)^2 / (n * (x_1^2 + x_2^2 + \dots + x_n^2))$$

xi는 사용자 i의 자원량, n은 사용자 수이다. 모두 동일 배분이면 1, 한 명이 독점하면 1/n에 가까워진다(예: 4명 중 한 명 독점 시 0.25). 스케줄링 맥락에서는 xi 자리에 사용자별 대기시간(값이 작을수록 좋음)이나 처리 건수(값이 클수록 좋음) 같은 값을 넣어 공정성을 측정한다. 실험 목적에 따라 두 기준을 구분해 사용하였다 (5.5절). 공정성 지표는 JFI 외에도 최대-최소 공정성(Max-Min Fairness, 가장 불리한 사용자의 몫을 극대화하는 기준)이나 비례 공정성(Proportional Fairness, 각 사용자의 만족도를 종합적으로 높이는 기준) 등이 있다 [4]. 그 가운데 계산이 간단하고 단일 수치로 비교하기 좋다는 점에서 JFI를 골랐다. 다만 WFQ처럼 가중치로 일부러 차이를 두는 방식에서는 단순 처리 건수로 JFI를 계산하면 "가중치대로 처리되었는가"를 판단할 수 없다. 그래서 xi 자리에 "처리 건수 ÷ 가중치"를 넣어 계산한 정규화(가중치 차이를 상쇄해 비교 기준을 맞추는 것) JFI를 사용한다. 본 보고서의 WFQ에도 이 방식을 적용하였다 (5.5절).

이러한 이론적 토대 위에서, 다음 장에서는 시스템 설계를 다룬다.

3. 시스템 설계

3.1 설계 개요

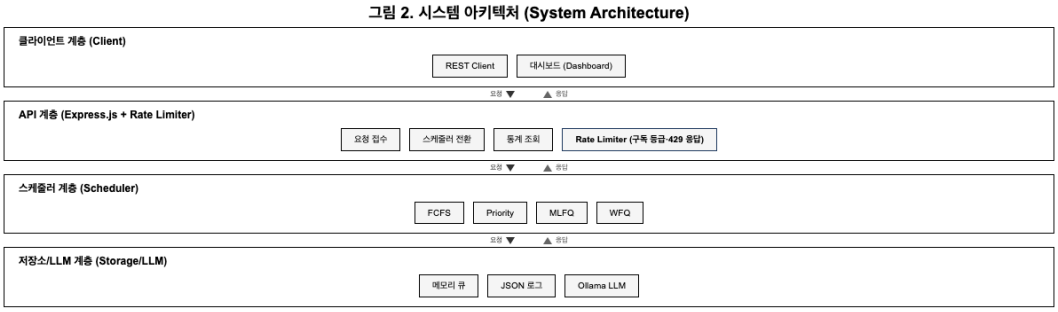
본 시스템의 기본 아이디어는 OS의 프로세스 준비 큐 구조에서 가져왔다. 프로세스가 CPU를 할당받기 위해 준비 큐에서 대기하듯, LLM API 요청도 대기열에서 순서를 기다리는 구조로 두었다 (표 1).

표 1. OS 개념과 LLM 환경 대응 관계

OS 개념	LLM 환경 대응	설명
프로세스	LLM API 요청	스케줄링 기본 단위
CPU 시간	LLM 추론 시간	할당되는 자원
준비 큐	요청 대기열	처리 대기 작업 저장소
우선순위	요청 우선순위	개별 요청의 긴급도 (Priority)
사용자 분류	구독 등급	서비스 수준 (WFQ, Rate Limiter)
시간 할당량	큐별 처리 시간 한도	MLFQ의 큐 이동 기준

3.2 시스템 아키텍처

시스템은 4개 계층으로 구성하였다 (그림 2). **클라이언트 계층**은 REST API(HTTP 요청으로 자원을 주고받는 웹 통신 방식)로 요청을 보내고 상태를 조회한다. **API 계층**은 Express.js [8] 기반 HTTP 서버이며, 요청을 받고 스케줄러를 전환하고 통계를 제공한다. 도착한 요청은 객체로 만들어진 뒤 Rate Limiter를 거쳐 스케줄러에 전달된다. **스케줄러 계층**에서는 네 알고리즘이 공통 인터페이스(enqueue/dequeue)를 구현하며, 환경변수로 선택된 알고리즘에 따라 처리 순서가 정해진다. **저장소/LLM 계층**은 메모리 큐로 요청을 관리하고 JSON 파일로 처리 이력을 기록한다. 실제 LLM 호출은 로컬의 Ollama로 수행한다.



[그림 2] 시스템 아키텍처

3.3 등급 체계

시스템에서 사용하는 분류 체계는 두 가지이며 각각 다른 목적으로 사용된다 (표 2).

표 2. 분류 체계 비교

구분	요청 우선순위 (Request Priority)	구독 등급 (Subscription Tier)
적용 대상	개별 요청	사용자(테넌트)
등급	URGENT, HIGH, NORMAL, LOW	Enterprise, Premium, Standard, Free
성격	요청마다 달라질 수 있음	사용자에게 고정
사용처	Priority Scheduling	WFQ 가중치, Rate Limiter

두 개념은 서로 독립적이어서 Enterprise 사용자가 LOW 우선순위 요청을 보낼 수도 있다. FCFS와 MLFQ는 우선순위·등급과 무관하게 동작한다.

3.4 설계 방침

Ollama를 비롯한 대부분의 LLM 서빙 도구가 추론 중단을 지원하지 않는다. 그래서 본 시스템도 비선점형(Non-preemptive, 실행 중인 요청을 끝까지 처리하고 중단하지 않는 방식)으로 동작하도록 구현하였다. 다만 MLFQ의 이론적 효과를 확인하려고 선점형(Preemptive, 할당량 소진 시 실행 중인 요청을 중단하고 다른 요청으로 교체하는 방식, 500ms 간격 할당량 확인)은 시뮬레이션에만 추가 구현하였다 (5.3절). 각 알고리즘의 세부 설정은 다음과 같다.

- **Priority**: URGENT 〉 HIGH 〉 NORMAL 〉 LOW 순, 5초 주기 에이징 적용
- **MLFQ**: 4단계 큐(Q0~Q3), 5초 주기 부스팅 적용 [3]
- **WFQ**: 구독 등급 가중치(Enterprise 100, Premium 50, Standard 10, Free 1)를 VFT 계산에 반영. 가중치 값은 Free를 기준으로 10배·50배·100배 차이를 두도록 실험을 위해 직접 정하였다.

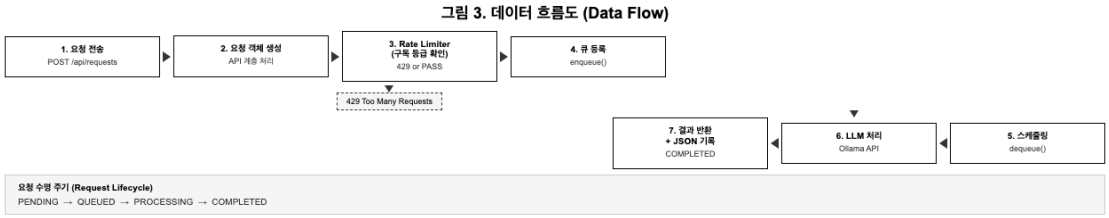
3.5 데이터 처리 흐름

사용자 요청은 다음 순서로 처리된다 (그림 3).

4. REST API(`POST /api/requests`)로 요청 도착
5. API 계층이 요청 객체 생성
6. **Rate Limiter가 등급별 한도를 확인** (초과 시 HTTP 429로 거부, 4.4절 참조)

7. 스케줄러 대기열에 등록(`enqueue`)
8. 스케줄러가 알고리즘에 따라 `dequeue`
9. Ollama에 전달해 응답 수신
10. 클라이언트 반환 및 JSON 이력 기록

대기열이 남으면 5번부터 반복한다.



[그림 3] 데이터 흐름도

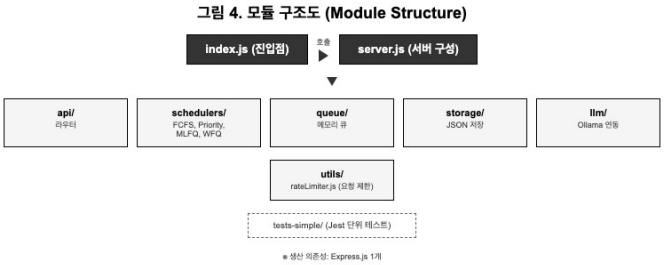
4. 구현

4.1 기술 스택

런타임은 Node.js 22 LTS [9], 웹 프레임워크는 Express.js 4.18 [8], 언어는 JavaScript(ES2024)를 사용하였다. LLM 실행에는 로컬 Ollama [10]를 썼고, 저장소는 메모리 큐와 JSON 파일로 구성하였다. 외부 라이브러리 의존성은 express 1개뿐이다. 외부 라이브러리를 최소화하고 스케줄링 알고리즘을 직접 구현해 학습 효과를 높이는 것이 목표였다. Ollama의 순차 처리·비중단 특성은 비선점형 설계와 잘 맞아 실서버 실험에 사용하였다 (5.4절).

4.2 코드 구조와 인터페이스

소스 코드는 핵심 모듈과 보조 요소로 구성된다 (그림 4). 핵심 모듈은 다섯 가지로, api(REST 라우터), schedulers(4개 알고리즘), queue(메모리 대기열), storage(JSON 기록), llm(Ollama 호출)이다. 보조 요소에는 index.js(메인 진입점), server.js(Express 서버 구성 모듈), rate-limiter가 있다. `npm start`로 실행하면 index.js가 server.js의 `createServer()`를 호출해 Express 서버를 띄운다.



[그림 4] 모듈 구조도

모든 스케줄러는 BaseScheduler를 상속받아 `enqueue(request)`·`dequeue()` 두 메서드를 공통 구현한다. 환경변수(`SCHEDULER_TYPE`)만으로 알고리즘을 전환할 수 있다(예: `SCHEDULER_TYPE=wfq`).

4.3 핵심 알고리즘 구현

FCFS·Priority는 단순하므로 구현이 복잡한 MLFQ와 WFQ를 중심으로 설명한다.

MLFQ 구현. OSTEP [3]의 5가지 규칙에 따라 구현하였다 (표 3).

표 3. MLFQ 규칙 및 큐별 시간 할당량

규칙	설명	큐	시간 할당량
Rule 1	높은 큐가 낮은 큐보다	Q0	1,000ms

	먼저 처리		
Rule 2	같은 큐 안에서는 도착 순서대로	Q1	3,000ms
Rule 3	새 요청은 Q0로 삽입	Q2	8,000ms
Rule 4	할당량 소진 시 하위 큐로	Q3	무제한
Rule 5	일정 주기로 모두 Q0로 부스팅		

dequeue는 Rule 1을 따라 상위 큐부터 FIFO(First-In, First-Out, 선입선출 — 먼저 들어온 요청부터 처리) 방식으로 꺼낸다.

```
// MLFQ 스케줄러: 다음 처리할 요청 선택 (비선점형)
dequeue() {
  for (let i = 0; i < 4; i++) {
    if (this.queues[i].length > 0) return this.queues[i].shift();
  }
  return null;
}
```

위 코드는 실서버에도 적용되는 비선점형 구현이다. 선점형 MLFQ는 시뮬레이션 전용으로 `run-multi-seed.js`에 따로 구현하였다. 500ms 간격으로 실행 시간을 확인해 할당량 초과 시 하위 큐로 옮기며, 5초 주기 부스팅으로 하위 큐의 기아를 막는다.

WFQ 구현. WFQ는 각 요청의 가상 완료 시각(VFT)을 계산해 가장 작은 값부터 처리한다.

VFT_i = VFT_(i-1) + 처리 비용 / 가중치

처리 비용은 요청 1건당 1로 고정하였다. LLM 출력 토큰 수는 생성 완료 전 알 수 없고, JFI 계산도 처리 건수 기준이므로 측정 기준을 일치시키기 위해서이다.

```
// WFQ: VFT 계산 후 enqueue
enqueue(request) {
  const tenant = this.tenants.get(request.tenantId);
  const startTime = Math.max(tenant.virtualTime, this.globalVirtualTime);
  request.virtualFinishTime = startTime + (1 / tenant.weight);
  tenant.virtualTime = request.virtualFinishTime;   // 다음 요청의 기준이 됨
  this.queue.push(request);
}
// VFT가 가장 작은 요청 선택
dequeue() {
```

```
    let minIdx = 0;
    for (let i = 1; i < this.queue.length; i++) {
      if (this.queue[i].virtualFinishTime < this.queue[minIdx].virtualFinishTime) minIdx = i;
    }
    const req = this.queue.splice(minIdx, 1)[0];
    this.globalVirtualTime = req.virtualFinishTime;   // 시스템 가상 시각 갱신
    return req;
  }
```

가중치가 큰 사용자는 VFT 증가분이 작아 자주 선택된다. Enterprise(가중치 100)와 Free(가중치 1)가 동시에 요청하면 VFT 증가분이 0.01 대 1로 Enterprise가 먼저 처리된다.

4.4 Rate Limiter

Rate Limiter는 스케줄링 비교 대상이 아니라 요청 진입을 관리하는 전처리 단계이며, 스케줄러와 독립적으로 동작한다. 토큰 버킷(Token Bucket, 일정 시간마다 요청 권한인 토큰이 새로 발급되고 요청이 들어올 때마다 토큰 하나씩 소모하는 방식) 방식으로 만들었다. 구독 등급별 분당 허용 수(표 4)를 제한하고, 토큰이 없으면 HTTP 429(Too Many Requests, 요청 한도 초과를 알리는 표준 응답 코드) [11]로 거부한다.

표 4. 구독 등급별 요청 한도

구독 등급	분당 허용 요청 수
Enterprise	100 req/min
Premium	50 req/min
Standard	10 req/min
Free	5 req/min

4.5 구현 결과 요약

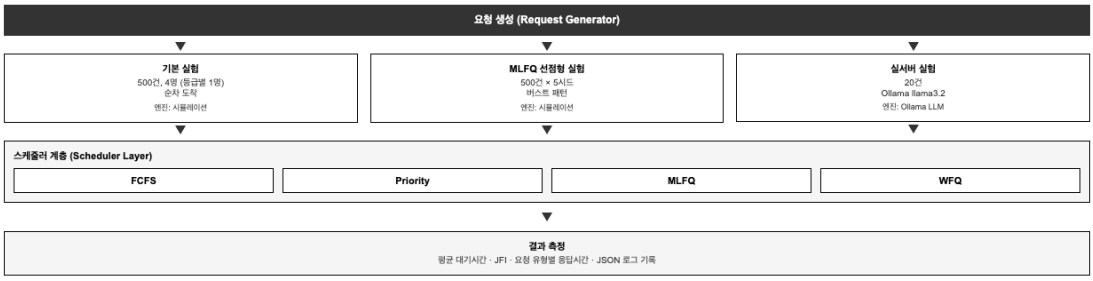
스케줄러 4종과 Rate Limiter를 모두 구현해 동작을 확인하였고, 스케줄러별 단위 테스트도 작성·통과시켰다. 환경변수(`SCHEDULER_TYPE`)만 바꾸면 알고리즘을 즉시 전환할 수 있다. 대기열 상태와 처리 통계를 브라우저에서 확인할 수 있도록 웹 대시보드(`public/dashboard.html`)도 함께 만들었다. 동일 요청을 알고리즘별로 어떻게 처리하는지 눈으로 비교할 수 있다.

5. 실험 및 평가

5.1 실험 환경

실험은 세 갈래로 나뉜다. 기본 실험, MLFQ 선점형 실험, 그리고 실서버 실험이다. 전체 구조는 그림 5, 실험별 주요 설정은 표 5에 정리하였다. 앞의 두 실험은 시간 기반 시뮬레이션이고, 실서버 실험은 Ollama로 실제 LLM을 호출해 진행하였다. 시뮬레이션은 요청에 처리 시간을 미리 지정해두고 스케줄러가 결정한 순서대로 한 건씩 처리하는 방식이다. 네트워크 지연이나 LLM 상태 변동이 없어 알고리즘 자체의 특성만 비교할 수 있다. 반복 실험에는 고정 시드(Seed, 난수 생성의 초기값이며 같은 시드로 실행하면 같은 요청 패턴을 다시 만들 수 있다)를 써서 같은 결과를 다시 얻도록 하였다.

그림 5. 실험 환경 구성도



[그림 5] 실험 환경 구성도

표 5. 실험 유형별 설정 비교

항목	기본 실험	MLFQ 선점형 실험	실서버 실험
목적	4개 알고리즘 동일 조건 비교	MLFQ 선점형 효과 검증	실제 LLM 환경 검증
처리 방식	시뮬레이션	시뮬레이션	Ollama 실제 호출
요청 수	500건	500건 × 5시드	20건
사용자 구분	4명 (등급별 1명)	미사용 (요청 유형만 구분)	4명 (등급별 1명)
처리 시간	10~100ms (균일 랜덤)	혼합 (Short/Medium/Long)	실제 LLM 추론 시간
도착 패턴	순차 도착 (1~10ms 간격)	버스트 (20건 동시 × 25회)	버스트 (20건 동시)
MLFQ 모드	비선점형	선점형 (500ms 간격)	비선점형
비교 대상	FCFS, Priority,	FCFS vs	FCFS, Priority, WFQ

	MLFQ, WFQ	MLFQ(선점형)	
재현성	고정 시드(42)	5개 랜덤 시드	동일 프롬프트 사용

선점형 실험의 "혼합"은 세 가지 처리 시간 범위의 요청을 섞은 것이다 (표 6).

표 6. 요청 유형별 구성

요청 유형	처리 시간 범위	비율	MLFQ 동작
Short	100~800ms	40%	Q0(1,000ms) 안에 완료
Medium	1,200~4,000ms	40%	Q0 초과, Q1 필요
Long	5,000~10,000ms	20%	Q1 초과, Q2 이상 필요

Short는 최상위 큐에서 완료되고 Medium·Long은 시간 할당량 초과로 하위 큐로 강등되도록 설계하였다. 버스트(Burst, 짧은 시간에 여러 요청이 동시에 몰리는 상황) 도착 패턴은 대기열에 요청이 쌓인 상태에서 스케줄러가 순서를 결정하도록 유도한다. 이를 위해 20건씩 동시에 투입하였다. 모든 실험에서 요청은 1건씩 순차 처리된다. JFI 계산 방식의 차이는 5.5절에서 다루며, 대규모 환경은 향후 연구 과제로 남긴다 (7.2절).

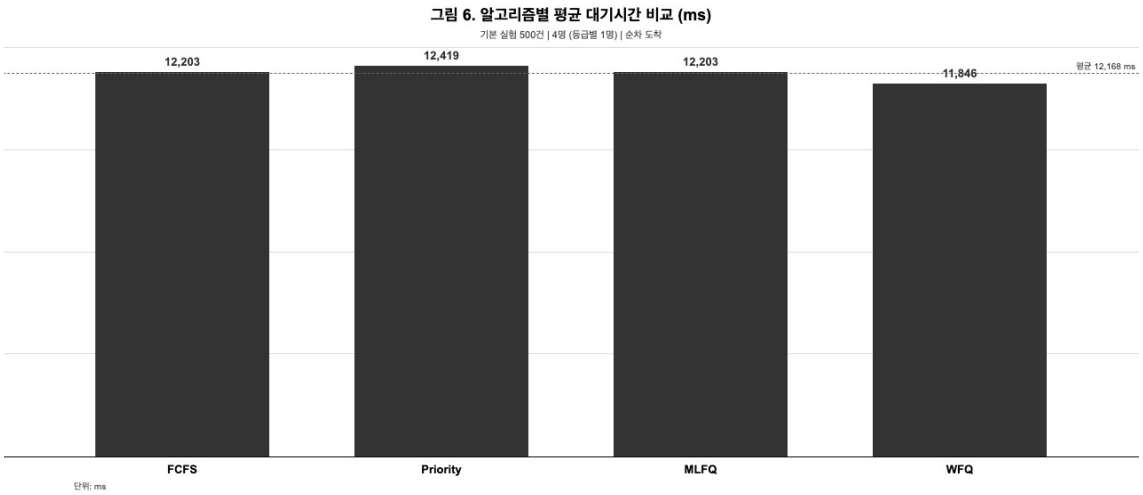
5.2 기본 실험 결과 (500건)

네 가지 알고리즘을 같은 500건 요청에 대해 실행한 결과는 표 7 및 그림 6과 같다.

표 7. 알고리즘별 성능 비교 (500건, 지표: 대기시간 기준)

스케줄러	평균 대기시간	처리량(req/s)	JFI
FCFS	약 12,203ms	약 18.72	1.000
Priority	약 12,419ms	약 18.56	1.000
MLFQ (비선점형)	약 12,203ms	약 18.72	1.000
WFQ	약 11,846ms	약 18.72	0.316

"대기시간(wait time)"은 요청이 대기열에 들어간 시점부터 처리가 시작된 시점까지이다. 단, 5.3절의 선점형 MLFQ 비교에서는 처리가 여러 번 중단·재개되므로 "응답시간(도착부터 완료까지)"으로 단위를 바꾼다.



[그림 6] 알고리즘별 평균 대기시간 비교

FCFS에서는 도착 순서대로 처리되어 네 명이 대기시간을 고르게 부담하였다 (JFI = 1.000). Priority는 처리 순서가 바뀌었지만 전체 평균은 FCFS와 거의 같았다(약 216ms 차이). 우선순위 큐는 단순 FIFO보다 삽입·추출에 약간 더 많은 계산이 들어가는데, 이 차이가 평균에 살짝 반영된 것으로 보인다. 거기에 처리 시간이 짧아(10~100ms) 에이징이 작동하기 전에 처리가 끝나서 우선순위 효과 자체는 거의 드러나지 않았다. MLFQ(비선점형)의 결과는 FCFS와 동일하였다. 모든 요청이 Q0 할당량(1,000ms) 안에 완료되어 하위 큐로 내려갈 일이 없었고, 비선점형 특성상 요청이 도착 순서대로 처리된 탓이다. 이론적 효과를 따로 확인하려고 5.3절에서 선점형 실험을 추가하였다. WFQ는 전체 평균이 가장 낮았으나 구독 등급별 차이가 컸다 (표 8). JFI = 0.316은 5.5절에서 분석한다.

표 8. WFQ 구독 등급별 평균 대기시간

구독 등급	가중치	평균 대기시간
Enterprise	100	약 1,862ms
Premium	50	약 8,384ms
Standard	10	약 15,109ms
Free	1	약 22,029ms

Enterprise의 대기시간은 Free의 약 1/12 수준으로, 가중치에 비례한 차등 서비스가 확인되었다. 100:1의 극단적 차이까지 벌어지지 않은 것은 요청이 대기열에 순차 도착하는 실험 설계 때문이다.

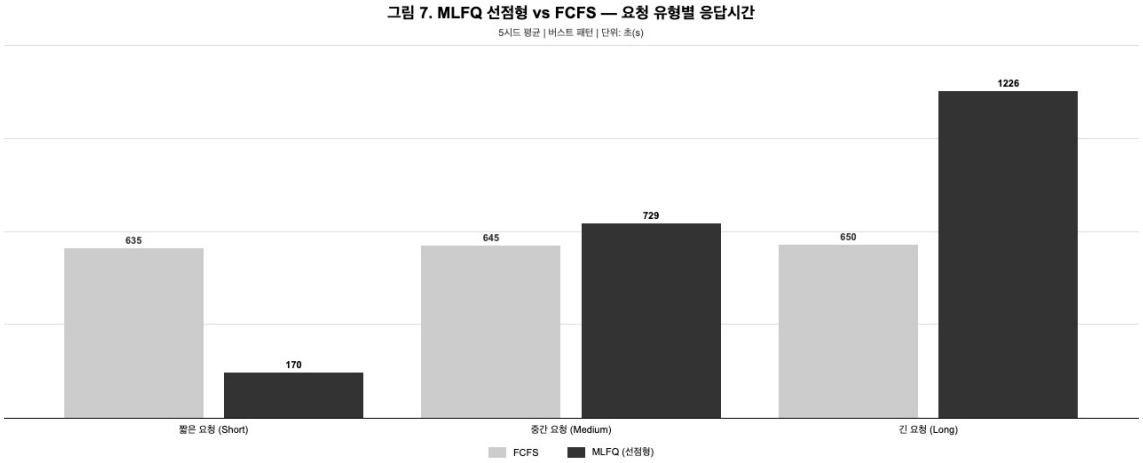
5.3 MLFQ 선점형 실험

비선점형 MLFQ가 FCFS와 동일한 결과를 보였기에, MLFQ의 이론적 효과를 따로 검증하려고 선점형 시뮬레이션을 추가하였다. 요청 500건, 처리 시간 최대 10초, 버스트 도착 구조로 5개 시드에 걸쳐

반복하였다. 지표는 응답시간(도착부터 완료까지)을 사용한다. 요청 유형별 응답시간 비교 결과는 표 9 및 그림 7과 같다.

표 9. MLFQ 선점형 vs FCFS 비교 (요청 유형별, 5시드 평균, 지표: 응답시간 기준)

요청 유형	처리 시간 범위	FCFS 응답시간	MLFQ (선점형) 응답시간	변화
Short	100~800ms	약 635초	약 170초	약 73% 감소
Medium	1,200~4,000ms	약 645초	약 729초	약 13% 증가
Long	5,000~10,000ms	약 650초	약 1,226초	약 89% 증가



[그림 7] MLFQ 선점형 vs FCFS 요청 유형별 응답시간

Short는 Q0 안에 완료되어 강등 없이 처리되므로 응답시간이 FCFS 대비 약 73% 감소하였다. 반면 Long은 하위 큐로 밀려 약 89% 증가, Medium은 약 13% 증가하였다. 부스팅(Rule 5, 5초 주기)으로 Long의 기아 상태는 방지된다.

재현성 확인: 5개 시드에서 Short 개선율이 71.8~74.3%의 좁은 범위에 모였다. 평균은 73.2%, 95% 신뢰구간(Confidence Interval, 같은 실험을 반복했을 때 참값이 이 범위 안에 있을 확률이 95%인 구간)은 [71.9%, 74.5%]였다. 쌍체 t-검정(Paired t-test, 같은 실험 조건에 두 방식을 적용하고 차이의 평균이 0인지 확인하는 통계 방법) 결과 $t = 45.59$, $p < 0.001$로 FCFS와 MLFQ의 Short 응답시간 차이는 통계적으로 유의하였고, 효과 크기 Cohen's $d = 20.39$로 매우 큰 차이가 일관되게 나타났다. 시드를 바꿔 다섯 번 반복해도 비슷한 개선율이 나왔다는 점은 이 효과가 특정 시드에 우연히 기댄 결과가 아니라는 뜻이다. 다만 5개 시드의 제한된 반복이라 실제 트래픽 패턴에서는 효과가 달라질 수 있다. 시드별 세부와 통계 계산 결과는 부록 B에 정리하였다.

5.4 실서버 실험

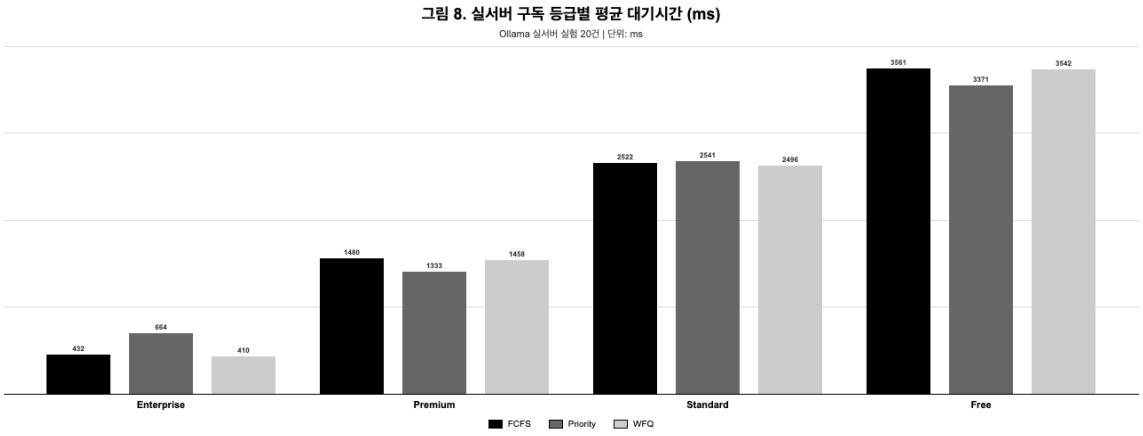
시뮬레이션에서 확인한 스케줄링 순서 차이가 실제 LLM 환경에서도 재현되는지 보고자 Ollama로 실서버 실험을 수행하였다. 사용 모델은 2026년 4월 공개된 Google Gemma 4 계열의 `gemma4:e4b`이며, 약 40억 개 파라미터(Parameter, 모델이 학습해 가진 가중치 값)를 가진 중형 모델로 로컬 Ollama 서버에서 실행하였다. 출력은 최대 20토큰으로 제한하였다. 총 20건(등급별 5건)을 동시에 큐에 투입한 뒤 FCFS, Priority, WFQ 세 알고리즘을 비교하였다(비선점형 MLFQ는 5.2절에서 FCFS와 동일함이 확인되어 제외하였다). 전체 결과는 표 10, 구독 등급별 세부는 표 11 및 그림 8에 정리하였다.

표 10. 실서버 실험 결과

스케줄러	평균 대기시간	평균 처리 시간	전체 소요 시간
FCFS	약 1,998ms	약 209ms	약 4,178ms
Priority	약 1,977ms	약 208ms	약 4,154ms
WFQ	약 1,976ms	약 208ms	약 4,167ms

표 11. 실서버 구독 등급별 평균 대기시간

구독 등급	FCFS	Priority	WFQ
Enterprise	약 432ms	약 664ms	약 410ms
Premium	약 1,480ms	약 1,333ms	약 1,458ms
Standard	약 2,522ms	약 2,541ms	약 2,496ms
Free	약 3,561ms	약 3,371ms	약 3,542ms



[그림 8] 실서버 구독 등급별 평균 대기시간 비교

세 알고리즘의 평균 처리 시간이 모두 약 208~209ms로 거의 같았다. 스케줄러는 "누구를 먼저 처리할지"만 결정할 뿐 LLM 추론 속도를 바꾸지 못한다. FCFS에서는 도착 순서대로 처리되어 Enterprise → Free 순으로 대기시간이 늘었다(Enterprise를 먼저 투입). Priority의 경우 구독 등급과 요청 우선순위가 독립적임을 드러내고자 등급별로 서로 다른 우선순위를 혼합 배정하였다(배정 내역은 부록 표 B.3 참조). 그 결과 Enterprise의 NORMAL 요청이 Premium의 URGENT 요청에 밀려 Enterprise 평균 대기시간은 FCFS 대비 증가하였다(432 → 664ms). 반면 Free의 NORMAL 요청은 Standard의 LOW 요청보다 먼저 처리되어 Free 평균 대기시간은 오히려 감소하였다(3,561 → 3,371ms). WFQ는 가중치에 따라 Enterprise가 먼저 처리되었다. 요청 수가 적어 등급별 극단적 차이까지는 드러나지 않았으나 시뮬레이션과 동일한 패턴이 나타났다.

5.5 공정성 분석

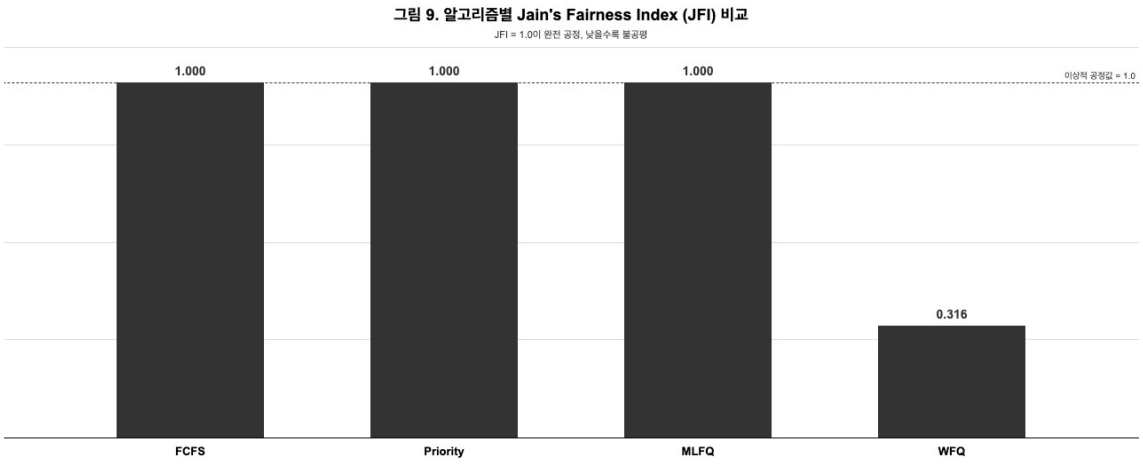
JFI 계산 방식의 차이부터 짚어둔다. 본 분석에서는 두 가지 기준을 실험 목적에 맞게 구분해 사용하였다.

- FCFS, Priority, MLFQ: 등급 간 대기시간 차이 측정 → **대기시간 기반** JFI (값이 1에 가까울수록 균등)
- WFQ: 가중치 비례 처리량 배분 측정 → WFQScheduler 내장의 **처리량 기반** JFI (가중치에 정확히 비례하면 1에 가까워짐)

표 12 및 그림 9의 0.316과 1.000은 계산 기준이 달라 직접 비교하지 않는다. 각 값의 의미는 아래에서 따로 해석한다.

표 12. 알고리즘별 JFI 비교 (기본 실험, 500건)

스케줄러	JFI	계산 방식	해석
FCFS	1.000	대기시간 기반	완전 공정 (도착 순서대로 처리)
Priority	1.000	대기시간 기반	사용자 간 공정 (우선순위는 요청 단위)
MLFQ (비선점형)	1.000	대기시간 기반	완전 공정 (FCFS와 동일)
WFQ	0.316	처리량 기반	의도적 차등 (가중치 기반)



[그림 9] 알고리즘별 Jain's Fairness Index(JFI) 비교

FCFS·Priority·MLFQ(비선점형)의 JFI 1.000은 세 알고리즘이 구독 등급을 고려하지 않기 때문이다. Priority는 처리 순서는 바뀌지만 우선순위가 요청 단위로 배정되어 사용자 간 차이는 없다.

WFQ의 공정성은 두 관점에서 본다. 첫째, **처리량 기반 JFI = 0.316**은 WFQScheduler가 "처리 요청 수 ÷ 가중치"로 계산한 값이다. 사용자당 125건씩 동일 배정했으므로 Free(가중치 1)의 비율이 커지고 Enterprise(100)는 작아져 JFI가 1에서 멀어진다. 이 값 자체가 WFQ가 가중치를 반영해 처리했다는 증거가 된다. 둘째, 등급 간 대기시간을 보면 Enterprise(약 1,862ms)가 Free(약 22,029ms) 대비 약 12배 짧아 가중치 비율(100:1)에 대응한다. 등급당 1명만 배치한 구성이라 같은 등급 내 공정성은 향후 연구 과제로 남긴다.

5.6 결과 종합

앞서 제시한 실험 결과를 종합하면 네 알고리즘의 처리 순서 기준, 성능 특성, 공정성 양상을 표 13과 같이 한눈에 비교할 수 있다.

표 13. 알고리즘 특성 비교 종합

특성	FCFS	Priority	MLFQ (선점형)	WFQ
처리 순서 기준	도착 순서	요청 우선순위	큐 레벨 (실행 시간)	구독 등급 가중치
짧은 요청 우대	없음	없음	있음 (약 73% 개선)	없음
차등 서비스	없음	요청 단위	없음	구독 등급 단위
JFI	1.000	1.000	1.000 (비선점)	0.316 (의도적 차등)
기아 방지	불필요	에이징	부스팅	VFT 구조상

				불필요
실서버 적용	가능	가능	비선점형만 가능	가능

6. 논의

6.1 시뮬레이션 환경의 한계

본 시뮬레이션은 알고리즘 특성을 비교하기에는 적합하지만, 실제 LLM 서빙 환경과 비교하면 몇 가지 한계가 있다. 첫 번째 한계는 **선점형 MLFQ를 실서버에 적용하기가 어렵다**는 점이다. Ollama를 비롯한 대부분의 LLM 서빙 도구는 추론 중단을 지원하지 않으므로, 5.3절에서 확인한 Short 약 73% 개선은 시뮬레이션 안에서만 가능하다. 다음으로 처리 시간 분포가 실제와 다르다. 기본 실험은 10~100ms 범위였으나, 실제 LLM 추론은 입력 길이·토큰·모델 크기에 따라 수백 ms에서 수십 초까지 폭이 넓고, 길어질수록 Priority 에이징 효과나 FCFS 호위 효과의 영향이 커진다. 또한 동시 처리도 고려하지 못하였다. 한 번에 1건씩 처리하였으나 실제 LLM 서빙(vLLM, TGI)은 연속 배치로 여러 요청을 동시 처리한다. 한 배치 내부에서는 순서가 영향을 주지 않으므로 WFQ·Priority의 차등 효과는 약해질 수 있다. 끝으로 실서버 실험 규모가 20건으로 제한적이다. 대기열 경합이나 극단적 트래픽은 재현하지 못하였으며, 실무에 적용하려면 대규모 부하 시험이 필요하다.

6.2 알고리즘별 적합 상황

- FCFS: 모든 사용자·요청을 동등하게 처리하는 가장 단순한 방식이다. 구현이 간단해 기본 정책으로 두기에 알맞다.
- Priority Scheduling: 긴급도를 직접 지정할 수 있는 환경에 적합하다. 다만 에이징 효과가 나타나려면 처리 시간이 어느 정도 길어야 한다.
- MLFQ: 짧은 요청 우대 효과가 뛰어나지만 선점형이 필요한 만큼, 현재 LLM 서빙 환경에서는 활용 범위가 좁다. 추론 중단·재개 기능이 추가된다면 활용 가능성이 열린다.
- WFQ: 구독 등급별 차등 배분이 필요한 유료 서비스에 적합하다. 비선점형에서도 잘 동작하여 실서버 적용이 가능하다.

6.3 프로젝트의 의의

이 프로젝트는 두 가지 면에서 의의가 있다. 우선 CPU 스케줄링(FCFS, Priority, MLFQ)과 네트워크 자원 배분(WFQ) 알고리즘을 LLM API 환경에 적용해 운영체제 이론을 실제 문제에 응용해 본 점이다. 더해서 vLLM·TGI 같은 기존 LLM 서빙 도구가 전체 처리량 최적화에 집중하느라 사용자별 공정성이나 우선순위 관리를 제공하지 않는다는 점을 확인하고, 그 빈자리에 스케줄링 이론을 적용해 알고리즘별 효과를 비교했다는 점도 의미가 있다. 특히 WFQ의 가중치 비례 차등 서비스는 유료 LLM 서비스의 구독 등급 관리에 쓸 수 있는 가능성을 보여준다. 다만 세 실험을 합산해도 총 3,020건 규모이므로, 대규모 운영 환경에서도 같은 결과가 나올지는 추가 확인이 필요하다.

7. 결론 및 향후 연구

7.1 결론

본 프로젝트에서는 운영체제의 네 가지 스케줄링 알고리즘(FCFS, Priority Scheduling, MLFQ, WFQ)을 다중 사용자 LLM API 요청 관리에 적용하고, 시뮬레이션 및 실서버 실험으로 성능과 공정성을 비교하였다.

RQ1 (스케줄링 적용): 네 알고리즘을 공통 인터페이스(enqueue/dequeue)로 구현하여 환경변수만으로 전환할 수 있게 하였다. MLFQ 선점형 모드는 실제 LLM 환경에서 적용이 어렵다는 한계를 확인하였다.

RQ2 (성능 비교): 비선점형 환경에서 네 알고리즘의 전체 처리량은 거의 같았다. 스케줄러는 순서만 결정할 뿐 처리 속도 자체를 바꾸지 못하기 때문이다. 선점형 MLFQ에서는 짧은 요청 응답시간이 약 73% 감소, 긴 요청은 약 89% 증가하였다. 개선 효과는 5개 시드에서 71.8~74.3% 범위로 일관되게 재현되었다 (시드별 세부는 부록 B). WFQ에서는 Enterprise의 평균 대기시간이 Free의 약 1/12 수준으로 나타났다.

RQ3 (공정성 분석): FCFS, Priority, MLFQ(비선점형)의 JFI는 모두 1.000으로 균등 배분이 이루어졌다. WFQ의 시스템 수준 JFI는 0.316이었으나, 이는 가중치 기반 의도적 차등의 결과이다. Enterprise의 대기시간이 Free보다 약 12배 짧아 가중치 순서에 대응하는 차등 서비스가 확인되었다.

또한 Rate Limiter를 통해 스케줄러와 독립적으로 과도한 요청을 사전 차단하는 기능을 구현하였다.

7.2 향후 연구

- **토큰 기반 비용 모델**: 현재 WFQ의 VFT 계산은 처리 비용을 요청 1건당 1로 고정하였다. 입출력 토큰 수를 반영한 가변 비용 모델을 적용하면 더 정밀한 자원 배분이 가능하지만, 출력 토큰 수 추정 방법에 대한 추가 연구가 필요하다.
- **대시보드 고도화**: 현재 대시보드는 주기적 조회(폴링) 방식으로 상태를 갱신한다. WebSocket(서버가 클라이언트에 실시간으로 알림을 보낼 수 있는 웹 통신 방식) 기반 푸시 갱신, 사용자 인증, JFI 추이 그래프 등을 더하면 운영 환경 모니터링 도구로 확장할 수 있다.
- **동일 등급 내 공정성 검증**: 이번 실험은 등급당 1명씩 배치한 구성이었다 (5.5절). 같은 등급에 여러 사용자가 있을 때의 균등 배분(등급 내 JFI)을 확인하려면 등급당 N명을 동시에 배치한 반복 실험이 필요하다.
- **대규모 실험**: 수천 건 이상의 요청, 더 많은 사용자, 더 긴 처리 시간 환경에서 각 알고리즘의 특성을 추가로 확인할 수 있다.

참고문헌

[1] OpenAI, "Rate limits," OpenAI API Documentation. [온라인] Available: https://platform.openai.com/docs/guides/rate-limits (접속: 2026-03-14)

[2] Anthropic, "Rate limits," Anthropic API Documentation. [온라인] Available: https://docs.anthropic.com/en/api/rate-limits (접속: 2026-03-14)

[3] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, "Operating Systems: Three Easy Pieces," v1.10, Arpaci-Dusseau Books, 2023. [온라인] Available: https://pages.cs.wisc.edu/~remzi/OSTEP/

[4] L. Peterson and B. Davie, "Computer Networks: A Systems Approach," 6th ed., Systems Approach LLC, 2022. [온라인] Available: https://book.systemsapproach.org/ (접속: 2026-04-19)

[5] vLLM Project, "vLLM: Easy, Fast, and Cheap LLM Serving," GitHub. [온라인] Available: https://github.com/vllm-project/vllm (접속: 2026-03-14)

[6] Hugging Face, "Text Generation Inference (TGI)." [온라인] Available: https://huggingface.co/docs/text-generation-inference/ (접속: 2026-03-14)

[7] Wikipedia, "Fairness measure -- Jain's fairness index." [온라인] Available: https://en.wikipedia.org/wiki/Fairness_measure (접속: 2026-04-09)

[8] Express.js, "Express - Node.js Web Application Framework." [온라인] Available: https://expressjs.com/ (접속: 2026-03-14)

[9] Node.js Foundation, "Node.js Documentation." [온라인] Available: https://nodejs.org/docs/latest-v22.x/api/ (접속: 2026-03-14)

[10] Ollama, "Ollama Documentation." [온라인] Available: https://ollama.com/ (접속: 2026-03-14)

[11] MDN Web Docs, "HTTP 429 Too Many Requests," Mozilla Developer Network. [온라인] Available: https://developer.mozilla.org/en-US/docs/Web/HTTP/Status/429 (접속: 2026-04-09)

부록

A. 소스 코드 파일 목록

표 A.1. 핵심 소스 코드 파일

파일 경로	역할
index.js	메인 진입점 (npm start 실행 대상)
server.js	Express 서버 구성 모듈
api/routes.js	REST API 라우팅
schedulers/BaseScheduler.js	enqueue/dequeue 공통 인터페이스
schedulers/FCFSScheduler.js	FCFS (FIFO)
schedulers/PriorityScheduler.js	Priority (에이징)
schedulers/MLFQScheduler.js	MLFQ (4단계 큐, 강등)
schedulers/WFQScheduler.js	WFQ (VFT, 가중치)
utils/rateLimiter.js	토큰 버킷 Rate Limiter
llm/OllamaClient.js	Ollama API 클라이언트

전체 소스는 `02-implementation/src-simple/`, 외부 의존성은 Express.js 1개이다. 실험 스크립트는 `02-implementation/experiments-simple/`에 있다.

표 A.2. 실험 스크립트 목록

파일	용도
run-extended.js	기본 실험 (500건, 4 스케줄러, 5.2·5.5절)
run-multi-seed.js	MLFQ 선점형 실험 (500건 × 5 시드, 5.3절)
run-ollama-experiment.js	실서버 실험 (20건, Ollama, 5.4절)
compute-stats.js	시드별 개선율 재현 확인 (부록 B)

B. 실험 구성 상세

표 B.1. MLFQ 선점형 실험 시드별 결과 (지표: Short 요청 응답시간)

시드	FCFS Short 응답시간	MLFQ Short 응답시간	개선율
12345	약 634,201ms	약 179,071ms	71.8%
23456	약 666,730ms	약 171,319ms	74.3%

34567	약 630,038ms	약 164,213ms	73.9%
45678	약 598,388ms	약 164,665ms	72.5%
56789	약 644,441ms	약 170,603ms	73.5%
평균	약 634,759ms	약 169,974ms	73.2%

다섯 개 시드 모두 70% 이상의 개선을 보였으며, 시드에 따른 개선율 편차는 약 2.5%p 범위에 머물렀다. 같은 시드 값으로 `compute-stats.js`를 실행하면 동일한 결과를 얻을 수 있다.

표 B.2. MLFQ 선점형 개선율 통계 요약 (지표: Short 요청 응답시간)

통계 지표	값
시드 수	5 (12345, 23456, 34567, 45678, 56789)
평균 개선율	73.2%
95% 신뢰구간	[71.9%, 74.5%]
쌍체 t-검정 t 값	45.59 (자유도 4)
p-value	〈 0.001
Cohen's d	20.39 (매우 큰 효과)

t-검정의 귀무가설(Null Hypothesis, "차이가 없다"고 가정하는 기본 전제)은 "FCFS와 MLFQ의 Short 요청 평균 응답시간이 같다"이다. 자유도(Degrees of Freedom, 독립적으로 변할 수 있는 값의 수로 표본 수에서 1을 뺀 값)는 5개 시드 표본에서 4가 된다. p 값이 0.001 미만이므로 두 방식의 평균 응답시간이 실제로 다르다고 볼 수 있다. Cohen's d = 20.39는 통상 "큰 효과"의 기준인 0.8을 크게 넘어서는 값으로, 두 방식의 응답시간이 거의 겹치지 않을 만큼 뚜렷한 차이를 보인다는 뜻이다. 시뮬레이션에서는 같은 요청 집합을 두 알고리즘에 각각 적용하므로 네트워크 지연 같은 외부 요인이 거의 없어 효과 크기가 이처럼 크게 나타났으며, 실제 운영 환경에서는 다양한 변동 요인 때문에 효과 크기가 더 작게 나타날 수 있다.

부스팅이 작동하는 기준 차이: 기본 실험(run-extended.js)은 "30건 처리마다", 선점형 실험(run-multi-seed.js)은 "5,000ms마다"이다. 기본 실험은 모든 요청이 Q0 안에서 완료되므로 부스팅이 실제로 적용되는 일은 없다.

선점형 시뮬레이션 해석: 이 표의 개선율은 LLM 추론을 원하는 때에 멈추고 다시 시작할 수 있다고 가정했을 때, 이론적으로 얻을 수 있는 최대 개선폭이다. 실서버에서는 비선점형 MLFQ만 적용할 수 있다 (6.1절).

표 B.3. Ollama 실서버 실험의 Priority 요청 우선순위 배정 (등급 × 요청 번호)

등급	요청 1	요청 2	요청 3	요청 4	요청 5
Enterprise	URGENT	HIGH	URGENT	HIGH	NORMAL

Premium	HIGH	NORMAL	HIGH	URGENT	NORMAL
Standard	NORMAL	LOW	NORMAL	HIGH	LOW
Free	LOW	NORMAL	LOW	LOW	NORMAL

상위 등급일수록 URGENT·HIGH 비율을 높이되, 모든 등급에 여러 우선순위가 섞이도록 배정하였다. 이렇게 한 이유는 구독 등급과 요청 우선순위가 서로 상관없는 두 기준임을 실험으로 확인하기 위해서이다. 이 배정에 따라 Priority 스케줄러에서는 Enterprise의 NORMAL 요청이 Premium의 URGENT 요청보다 뒤로 밀리고, Free의 NORMAL 요청이 Standard의 LOW 요청보다 앞당겨지는 현상이 5.4절에서 확인되었다.